# Deep Reinforcement Learning Based Latency Minimization for Mobile Edge Computing With Virtualization in Maritime UAV Communication Network

Ying Liu, Junjie Yan, *Student Member, IEEE*, and Xiaohui Zhao, *Member, IEEE*

***Abstract*—The rapid development of maritime activities has led to the emergence of more and more computation-intensive applications. In order to meet the huge demand for wireless communications in maritime environment, mobile edge computing (MEC) is considered as an effective solution to provide powerful computing capabilities for maritime terminals of resource scarcity or latency sensitive. A basic technology to implement MEC is virtual machine (VM) multiplexing, through which multi-task parallel computing on a server is realized. In this paper, a two-layer unmanned aerial vehicles (UAVs) maritime communication network with a centralized top-UAV (T-UAV) and a group of distributed bottom-UAVs (B-UAVs) is established and MEC is used on T-UAV. We aim to solve the latency minimization problem for both communication and computation in this maritime UAV swarm mobile edge computing network. We reformulate this problem into a Markov decision process (MDP), since it is a non-convex and multiply constrained but has the characteristics of MDP. Based on this MDP model, we take deep reinforcement learning (DRL) as our tool to propose a deep Q-network (DQN) and a deep deterministic policy gradient (DDPG) algorithms to optimize the trajectory of T-UAV and configuration of virtual machines (VMs). Using these two proposed algorithms, we can minimize the system latency. Simulation results show that the given solutions are valid and effective.**

***Index Terms*—Maritime communication, deep reinforcement learning (DRL), latency minimization, UAV trajectory design, mobile edge computing (MEC), virtual machine (VM).**

## I. Introduction

THE vast ocean territories on the earth and its huge associated economic potential have led to the rapid development of ocean activities in recent years [1]. In addition to traditional activities such as maritime transportation and fishery, there are important explorations of seabed resources, environmental monitoring [2], maritime tourism, maritime rescue [3], maritime military activities, etc. Ocean economy has increasingly become the top priority of all countries in the world. The rapid growth of these activities, and the development of the Internet of Things (IoT) applications result in the emergence of more and more computation-intensive applications, and meanwhile greatly promote the huge demand for wireless communications in maritime environment [4]–[6]. Therefore, it is necessary and significant for the study on maritime communication networks, wireless or optical, to improve the communication performance.

However, there is a big difference between the terrestrial and maritime communication environments. The terrestrial Internet can be readily accessed on land through various types of terminals, but currently, the maritime communication networks mainly use high frequency/very high frequency(HF/VHF) radio systems, on-shore cellular networks and various satellite communication systems [7]. HF and VHF communications have been extensively used for buoy-to-shore data transmission and ship-to-ship or ship-to-shore communications [8], respectively. They provide communication coverage of hundreds of kilometers at low cost, however due to the limited channel bandwidth, they cannot be used to transmit substantial data services like videos or great amount of information. Coastal users are available for terrestrial cellular networks and wireless fidelity (Wi-Fi) communications, but the on-shore infrastructures can only serve a small number of users due to the limited maximum transmission distance in about 100 kilometers [9], [10]. Satellite communications can provide a worldwide internet access for maritime terminals to meet the communication needs of high data rate and large coverage. At the time-being, they have the disadvantages of expensive cost and limited bandwidth due to their structure, launch and operation reasons [8]. In general, the current compositions of maritime communication network have certain limitations, we need more solutions for Big Data, high-speed and cost-effective maritime communication.

In addition, from the perspective of communication environment, maritime environment is very different from terrestrial environment in terms of geographic consideration (71% of surface occupation of the Earth), climatic conditions (high humidity and frequent extreme weather), and user distribution (transient and uneven distribution). These differences make it extremely difficult and expensive to deploy communication infrastructures, and produce unique communication characteristics, such as long communication distances and unstable channels due to the blockage caused by sea clutter [11].

Manuscript received August 10, 2021; revised November 8, 2021; accepted January 4, 2022. Date of publication January 11, 2022; date of current version May 2, 2022. This work was supported by the National Natural Science Foundation of China under Grant 61571209. The review of this article was coordinated by Prof. Bin Lin. *(Corresponding author: Xiaohui Zhao.)*

The authors are with the College of Communication Engineering, Jilin University, Changchun, Jilin 130012, China (e-mail: lying19@mails.jlu.edu.cn; yanjj19@mails.jlu.edu.cn; xhzhao@jlu.edu.cn).

Digital Object Identifier 10.1109/TVT.2022.3141799

In this context, mobile edge computing (MEC) is considered as an effective solution to provide powerful computing capabilities for maritime terminals of resource scarcity or latency sensitive. It can better meet the requirements of maritime users in terms of Big Data, low latency, low cost and high reliability. Different from the remote cloud center, a MEC server not only has powerful computing and storage capabilities, but also is close to the network edge with a short transmission distance, which can effectively reduce transmission latency and energy consumption [12]. Literature [13] shows that the latency of offloading tasks to edge nodes is about 13% less than those to cloud nodes. A basic technology to implement MEC is virtual machine (VM) multiplexing, that is, multiple VMs are allocated in a same physical machine (PM). Specifically, each VM is configured with a certain amount of hardware resources (such as CPU, main storage, and I/O bus), so that each MEC server can accommodate multiple computing tasks. Nevertheless, the authors in [14] find that sharing the same PM will cause a certain decline of computing-speed for each VM, mainly due to the I/O interference among VMs. Although I/O interference is an important issue in virtualization, it has been ignored in many literatures. In [15], [16], the authors define a degradation factor $D > 0$ as the percentage increase of the expected service time when a VM is multiplexed with other VMs. As far as we know, the previous researches on this issue focus on the parallel computation of tasks with the same amount of data in different VMs. There have been no works related to the parallel computation of tasks with different amount of data in different VMs. This paper will study the problem of parallel computation of tasks offloading with different amount of data in different VMs in a MEC maritime communication system with I/O interference.

In order to improve the performance of maritime communication network from different aspects, the combination of UAV and MEC is studied and used in recent years. Many researchers have achieved great progresses on the integration of UAV and MEC. By deploying edge servers in UAVs, edge computing performs to enhance the performance of maritime communication network [17]. As a mobile computing server, UAV can realize rapid deployment and flexible offloading of computing tasks in a maritime communication network. The advantages are in two aspects: (1) UAV with higher altitude can readily establish line-of-sight links with maritime terminals to potentially boost communication throughput. (2) flexibly deployed UAV can provide shorter transmission distance and faster response. Therefore, UAVs have been widely used in wireless communications due to their mobility, easy deployment and operability.

Based on the above discussions and motivated by the related previous researches, we conceive a two-layer UAV-enabled MEC network framework, including a centralized T-UAV and a group of distributed B-UAVs. We aim to meet sensitive latency requirements of maritime users by optimal trajectory design and number configuration of VMs. At the first layer of this system, maritime users offload their computation-intensive and latency-sensitive tasks to B-UAVs. Once the computing tasks in a certain area increase sharply, and the processing capacity of edge servers reaches saturation, which result in long task queue time and latency, the deployed T-UAV above the first layer will offload the B-UAVs tasks according to offloading decision strategy. Therefore, the coordination of T-UAV and B-UAVs as a supplementary offloading solution for central cloud can better reduce task latency and support real-time processing of maritime tasks, since offloading, parallel computing and downloading are three factors that cause latency. Compared with offloading time, downloading time for sending computation results back can be ignored. On the basis of the proposed system, we jointly optimize T-UAV trajectory and the number of VMs according to the queuing calculation data size and positions of B-UAVs to minimize the latencies for both communication and computation. The principal contributions are as follows.

1) Since the current composition of maritime communication network can not meet Big Data, high-speed and cost-effective communication needs, we use UAV-enabled MEC as an effective solution to provide powerful computing capabilities for resource-scarce or delay-sensitive maritime terminals. Considering limited energy and resources of UAVs, we propose a two-layer UAV maritime communication network architecture to optimize the wireless network performance in MEC scenario through resource sharing between UAVs. This collaborative UAV-enabled MEC network architecture, as a supplement for central cloud, provides efficient computing power and reduces task completion time under the limited resources.
2) Compared with the existing literatures which study the parallel computing of tasks with the same amount of data in different VMs, this communication model considers the parallel computing neglected in most edge computing. We propose a different parallel computing of tasks with different amount of data in different VMs in a MEC system with I/O interference. In practice, I/O interference causes the computing speed of each VM to drop to a certain extent, and the amount of data for each computing task is usually different. Considering this two problems, we optimize the number of VMs according to the amount of data for each computing task to minimize the parallel computing latency.
3) We formulate this non-convex optimization with different practical constraints into a MDP, which is impossible to solve by traditional convex optimization methods. After the formulation, we propose a DQN and DDPG algorithms respectively to minimize the system latency for the intensive computing tasks. Our DQN algorithm is easy to implement and suitable to the simple case with discrete or small dimension spaces, while our DDPG algorithm is for the case of large dimension or continuous spaces. The principal goal of the optimization is to find optimal flight trajectories and the number of VMs participating in parallel computing for T-UAV.

The remainder of this article is organized as follows. Related works are discussed in Section II. Section III introduces the system model. Section IV presents two RL-based algorithms for the minimization of the system latency. The simulation results are given in Section V. Finally, we conclude this article in Section VI.

## II. Related Works

In many literatures, in-depth research has been conducted on the problems of computing offloading, the UAV trajectory optimization and the resource allocation in UAV-enabled edge computing systems. The authors in [18] design a UAV-enabled edge computing system which minimizes the sum of maximum latency as an objective by jointly optimizing UAV trajectory, offloading task ratio, and user scheduling. They show that the mobile UAVs can effectively improve the computing performance. The authors in [19] adopt UAV-enabled MEC to offload computing tasks for ground users according to their requirements. And an optimization method for the UAV trajectory design and resource allocation is proposed to minimize energy consumption whit limited energy. Since the provision of high-speed computing or energy-saving mobile services is a major focus for the design of UAV-enabled MEC system, the authors in [18] and [19] emphasize the problem of minimizing latency and energy consumption, respectively. However, two results are both for a single UAV to provide offloading services for mobile users. A natural question is whether we can deploy multi-UAVs to serve many mobile users at the same time in a large area on sea. Compared with a single UAV, multi-UAVs can support more tasks in a shorter time and provide large-scale computing offloading services for different mobile users. In [20], the authors propose a low-complexity algorithm to solve a non-convex problem by three separated sub-problems iteratively. They reasonably allocate resources to minimize the total power consumption of the terminals and UAVs in a multi-UAV enabled MEC network. To deal with the non-convexity of an optimization problem and the coupling between different variables, the authors in [21] propose an iterative optimization algorithm with double-loop structure to jointly optimize the resource allocation and trajectory scheduling to maximize the computational efficiency in a multi-UAV enabled MEC system.

These above studies have made important contributions to the related fields based on MEC, but there are few references for MEC to improve the performance of maritime applications. Recently, hybrid satellite-UAV-terrestrial maritime communications based on MEC are one of the main research directions to solve maritime communication problems. The authors in [4], [5], and [26] discuss the applications of satellites and UAVs to provide edge computing services for maritime terminals. In a satellite network, only low-earth orbit satellites can provide services such as real-time data processing and caching for areas where ground base stations are not available in maritime communications. We know that satellite communication is more suitable for latency-tolerant, large-bandwidth and long-distance services [24]. Additionally, more and more maritime applications require powerful computing capabilities to meet Big Data, low latency, low cost and high reliability services, but long-distance satellite links cannot satisfy these requirements [21]. On the contrary, UAV-enabled MEC servers can well overcome the above shortcomings, since they have shorter communication latency, large sea coverage and more flexible deployment. Normally, we use UAV-enabled MEC servers to reduce energy consumption and transmission latency for maritime equipment, which will effectively address the challenges of high load and low latency in rapid development of maritime communication networks, and greatly improve maritime user experience. The researches on UAV-enabled MEC are mostly user-centric. There are only few studies on the UAV-centric with sharing resources among UAVs to optimize the performance of entire system in MEC scenario, because of limited energy and resources of UAVs. Most of the existing works is on the trajectory design for UAV-enabled communications by transforming an original non-convex optimization problem into a convex problem to solve. In fact, this method often simplifies the problem through converting continuous trajectory design into hovering design for UAVs.

Interestingly, some studies begin to model the UAV-enabled communication problem as a Markov decision process (MDP). They take reinforcement learning (RL) to interact with real working environment to find an optimal solution without non-convexity transformation. In [25], the authors model a UAV trajectory optimization problem into a MDP to maximize the throughput through a RL based algorithm without considering energy consumption and resource allocation. The authors in [26] propose a deep Q-network (DQN) algorithm which uses deep neural networks (DNN) to formulate action strategies and optimize the overall performance of the network. However, DQN cannot work well for continuous spaces because of huge dimension and may have overestimation problem. In order to solve this problem, the authors in [27] propose a double DQN algorithm which uses different value functions to implement action selection and action evaluation. However, when DQN is applied to continuous domain, it may suffer from dimension disaster of continuous space. For these considerations, the authors in [28] propose a deep deterministic policy gradient (DDPG) algorithm in order to apply deep reinforcement learning (DRL) to the continuous action space. It is proved that this algorithm can optimize the flight direction of UAV and the bandwidth allocated to each user to realize energy-efficient and fair communication. A effective and DDPG-based collaborative computation offloading and resource allocation framework of multi-agent RL has also been provided by [29].

The combination of MEC and machine learning (such as DRL) is considered as an essential component of the future IoT not only in 5G applications but also in future 6G scenarios. The authors in [30] introduce how to obtain, organize, manage and utilize local and global information through the aviation data lake (ADL). The data lake can be easily connected with deep learning (DL) and reinforcement learning (RL) methods for intelligent signal processing to provide various high-data-rate services. The authors in [31] propose a technology to achieve 6G is the DL based transmission prediction, which can actively predict user requests and time-varying channel conditions for reducing latency and ensuring reliability. Compared with 5G of scene-centric services, 6G focuses on providing secure wireless computing in the era of Big Data. With the development of data mining, edge computing, mobile caching, general computing has become an important service objective of wireless communications.

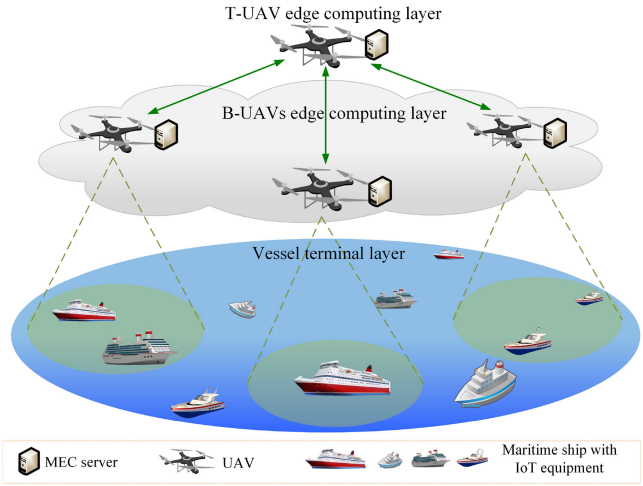


Fig. 1. System model.

All the above studies bring us lots of thinking and helpful results to our considered problem and proposed solutions.

## III. System Model

As shown in Fig. 1, a two-layer UAV-enabled MEC network is constructed to minimize system latency. In the first layer from maritime users to B-UAVs, B-UAVs perform as edge servers to provide offloading services for maritime users under their covered sea. Each B-UAV equipped with a computing processing unit and a fixed-beamwidth antenna hovers over its mission area [32]. For each B-UAV, we assume that maritime users are randomly distributed in these B-UAVs covered areas. We also assume that each maritime user has a computation-intensive and latency-sensitive task to be offloaded. Once the computing resources of B-UAVs in a certain area are insufficient and the computing tasks are delayed due to long queue time, an idle UAV in other areas will be deployed above the second layer, denoted as T-UAV in Fig. 1, to help B-UAVs to offload their tasks in the queue. In this paper, we mainly consider the second layer computation-offloading problem. The target is to minimize the communication latency and computation latency from B-UAVs to T-UAV by jointly optimize the trajectory control and the number of VMs configuration of T-UAV.

### A. T-UAV Trajectory

We assume that the necessary energy of all UAVs is large enough to complete their flight operations and wireless communications during a flight period of $T$. The flight period is discretized into $N$ time slots by equally and sufficiently small $\delta$. We define a set $\mathcal{N} = \{1, 2, \ldots, n, \ldots, N\}$ and assume that each UAV is at a constant position in time slot $\delta$. In our model, we consider that $M$ B-UAVs hover at a constant height $h$ and the horizontal coordinate of B-UAV $i$ is $\mathbf{q}_i^B = (x_i, y_i, h), i \in \mathcal{M}$, where set $\mathcal{M} = \{1, 2, \ldots, i, \ldots, M\}$. A T-UAV flies with a chosen trajectory to minimize the transmission latency of the system. The horizontal coordinate of T-UAV in time slot $n$ is $\mathbf{q}_n^T = (X_n, Y_n, H), n \in \mathcal{N}$. The flight direction of T-UAV is determined by the angle of $\theta \in [0, 2\pi)$, the distance of $l \in [0, l_{\max}]$, and the flight speed of $v_{\max}$ m/s, which means that the T-UAV flight distance is not more than $l_{\max} = v_{\max}\delta = v_{\max}\frac{T}{N}$ meters in a time interval [33]. We assume that the initial coordinate of T-UAV is $\mathbf{q}_0^T = (X_0, Y_0, H)$. Then, the coordinate $\mathbf{q}_n^T = (X_n, Y_n, H)$ of T-UAV in time slot $n$ can be calculated by $X_n = X_0 + \sum_{t=1}^{n} l_t \cos(\theta_t)$ and $Y_n = Y_0 + \sum_{t=1}^{n} l_t \sin(\theta_t)$. Hence, we have the coordinate constraints as the following

$$0 \le X_{n+1} \le X_n + l_{\max} cos(\theta_{n+1}), \forall n \in \mathcal{N} \tag{1}$$

$$0 \le Y_{n+1} \le Y_n + l_{\max} sin(\theta_{n+1}), \forall n \in \mathcal{N} \tag{2}$$

### B. Communication Uplink

We assume that the communication links between B-UAVs and T-UAV are dominated by wireless link with line-of-sight (LoS) characteristics where the channel quality only depends on communication distance [11], [34]. We denote the distance between T-UAV and B-UAV $i$ in time slot $n$ as $d_{i,n}$, expressed by

$$d_{i,n} = \sqrt{\left\|\mathbf{q}_n^T - \mathbf{q}_{i,n}^B\right\|^2}, \forall n \in \mathcal{N}, i \in \mathcal{M} \tag{3}$$

The channel power gain between T-UAV and B-UAV $i$ at time slot $n$ follows the free-space path loss model

$$h_{i,n} = \rho_0 d_{i,n}^{-2} = \frac{\rho_0}{\left\|\mathbf{q}_n^T - \mathbf{q}_{i,n}^B\right\|^2}, \forall n \in \mathcal{N}, i \in \mathcal{M} \tag{4}$$

where $\rho_0$ represents the channel power gain at a reference distance of 1 m, and it depends on the carrier frequency and antenna gains.

Within the time intervals, T-UAV serves all B-UAVs via frequency-division multiple access (FDMA) [35]. The whole system bandwidth $B$ is divided into $M$ non-overlapping sub-bands for $M$ B-UAVs on average, and the sub-bands $\frac{B}{M}$ is allocated to each B-UAV in each time slot. Then, the signal-to-noise ratio (SNR) of the date transmission from the $i$th B-UAV to T-UAV can be modeled as [36]

$$\begin{aligned} SNR_{i,n} &= \frac{h_{i,n} P_{i,n}}{\sigma_0^2 B\ M} \\ &= \frac{\rho_0 P_{i,n}}{\sigma_0^2 B\ M \left\|\mathbf{q}_n^T - \mathbf{q}_{i,n}^B\right\|^2}, \forall n \in \mathcal{N}, i \in \mathcal{M} \end{aligned} \tag{5}$$

where $P_{i,n}$ represents the transmission power of the $i$ th B-UAV, and $\sigma_0^2$ in watt/Hz denotes the power spectrum density of the white Gaussian noise (WGN) at T-UAV. According to the Shannon theorem, the uplink date rate of B-UAV $i$ in time slot $n$ is

$$R_{i,n} = \frac{B}{M} log_2\left(1 + \frac{\rho_0 P_{i,n}}{\sigma_0^2 B\ M \left\|\mathbf{q}_n^T - \mathbf{q}_{i,n}^B\right\|^2}\right), \forall n \in \mathcal{N}, i \in \mathcal{M} \tag{6}$$

### C. Offloading Time

When the computation-intensive and latency-sensitive tasks of the marine users in a certain area increase sharply, the task

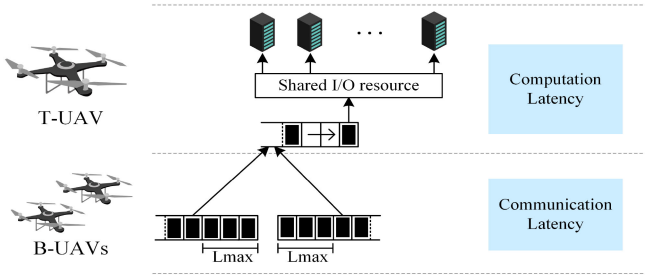


Fig. 2. VMs parallel computing latency.

requests will be enqueued in each B-UAV queue, which has a limited size $L_{\max}$. Once the limit is reached, the low-latency requirements of users will not be met, and their further requests will be rejected, and they have to wait in the queue and choose to offload to T-UAV. Therefore, whether B-UAV needs to offload its tasks or not has a crucial impact on the function and performance of the system.

We define $b_{i,n}$ and $L_{i,n} = \sum_{j=1}^{K_i} L_{i,j,n}, \forall i \in \mathcal{M}, n \in \mathcal{N}$, $j \in \mathcal{K}_i$, where $\mathcal{K}_i = \{1, 2, \ldots, j, \ldots, K_i\}$, as the offloading decision and the queued computation input data of B-UAV $i$, respectively. If $L_{i,n} \geq L_{\max}, b_{i,n} = 1, \forall i \in \mathcal{M}, n \in \mathcal{N}$, B-UAV $i$ offloads the over-waiting tasks outside the limited length of the queue to T-UAV through wireless network for execution. If $L_{i,n} < L_{\max}, b_{i,n} = 0, \forall i \in \mathcal{M}, n \in \mathcal{N}$, B-UAV $i$ continues to execute the waiting tasks within the queue. The offloading decision can be written as

$$b_{i,n} = \begin{cases} 1, \text{ offloading to T} - \text{UAV} \\ 0, \text{ executing on B} - \text{UAV} \end{cases} \tag{7}$$

According to these definitions, the uplink transmission time from the $i$th B-UAV to T-UAV in time slot $n$ is

$$\begin{aligned} t_{i,n} &= \frac{b_{i,n}\left(L_{i,n} - L_{\max}\right)}{R_{i,n}} \\ &= \frac{b_{i,n}\left(\sum\limits_{j=1}^{K_i} L_{i,j,n} - L_{\max}\right)}{R_{i,n}}, \forall i \in \mathcal{M}, j \in \mathcal{K}_i, n \in \mathcal{N} \end{aligned} \tag{8}$$

Thus, the uplink transmission time of the system in time slot $n$ is

$$\begin{aligned} T_n^{tran} &= \sum_{i=1}^{M} t_{i,n} \\ &= \sum_{i=1}^{M} \frac{b_{i,n}(L_{i,n} - L_{\max})}{R_{i,n}}, \forall i \in \mathcal{M}, n \in \mathcal{N} \end{aligned} \tag{9}$$

### D. Parallel-Computing With Virtualization

As shown in Fig. 2, when the B-UAV queue reaches $L_{\max}$, the newly arrived tasks will be offloaded from B-UAVs to T-UAV. After receiving all offloading tasks, T-UAV executes these received tasks in parallel by creating multiple virtual machines (VMs) on the same physical machine (PM) [37]. However, the multiple VMs sharing same PM will cause overall performance degradation mainly due to the I/O interference between these VMs. We define a degradation factor $D > 0$ as the percentage of the overall performance degradation of the multiple VMs when a VM is multiplexed with other VMs. We assume that a VM is assigned to a B-UAV, and when only one B-UAV needs to be offloaded, the expected completion service time in time slot $n$ is $T_{i,n} = \frac{b_{i,n}(L_{i,n} - L_{\max})}{r_{i,n}}, \forall i \in \mathcal{M}, n \in \mathcal{N}$, where $r_{i,n}$ is the computation rate (bits/sec) at which only a VM is running in isolation. When two B-UAVs need to be offloaded, such as B-UAV1 and B-UAV2, the corresponding expected computation service time of a VM in time slot $n$ running in isolation is $T_{1,n} + T_{2,n}$. According to [15], we can derive that the expected computation service time to execute two multiplexed VMs in time slot $n$ is $T_n^{comp} = T_{\max,n}(1 + D)$, where $T_{\max,n} = \max\{T_{1,n}, T_{2,n}\}$. Hence, the expected execution time of $S_n$ multiplexed VMs in time slot $n$ can express as

$$T_n^{comp} = T_{\max,n}\left(1 + D\right)^{S_n - 1}, n \in \mathcal{N} \tag{10}$$

where $T_{\max,n} = \max\{T_{1,n}, T_{2,n}, \ldots, T_{M,n}\}$ and $S_n$ represents the number of VMs of T-UAV in time slot $n$.

### E. Problem Formulation

The saturated B-UAVs can offload data packets to T-UAV, which will be delivered to different VMs via VM multiplexing for processing. Fig. 2 also shows the VMs parallel computing latency. When a data packet flows from a B-UAV to T-UAV, the latency can be divided into communication latency and computation latency [38]. Our objective is to minimize these two latencies by jointly optimizing the trajectory of T-UAV and the number of VMs configuration during flight period. Let $\mathbf{Q} = \{\mathbf{q}_n^T\}_{n \in \mathcal{N}}$ and $\mathbf{K} = \{S_n\}_{n \in \mathcal{N}}$ denote two sets of optimization variables related to UAV trajectory control and the number of VMs configuration, respectively. Thus, the total latency minimization problem is formulated as

$$\mathbf{OP1}: \min_{\mathbf{Q},\mathbf{K}} \sum_{n=1}^{N} (T_n^{tran} + T_n^{comp}) \tag{11}$$

$$s.t. \quad 0 \leq \theta < 2\pi \tag{11a}$$

$$0 \leq l \leq l_{\max} \tag{11b}$$

$$0 \leq X_{n+1} \leq X_n + l_{\max} cos(\theta_{n+1}), \forall n \in \mathcal{N} \tag{11c}$$

$$0 \leq Y_{n+1} \leq Y_n + l_{\max} sin(\theta_{n+1}), \forall n \in \mathcal{N} \tag{11d}$$

$$b_{i,n} \in \{0, 1\} \tag{11e}$$

$$b_{i,n} = \begin{cases} 1, \text{ if } L_{i,n} \geq L_{\max} \\ 0, \text{ if } L_{i,n} < L_{\max} \end{cases}, \forall i \in \mathcal{M}, n \in \mathcal{N} \tag{11f}$$

$$0 \leq S_n \leq \sum_{i=1}^{M} b_{i,n}, \forall n \in \mathcal{N}, i \in \mathcal{M} \tag{11g}$$

where $L_{i,n} = \sum_{j=1}^{K_i} L_{i,j,n}$ and $l_{\max} = v_{\max} \frac{T}{N}$ represent the queued computation input data of B-UAV $i$ and the maximum flight distance of T-UAV in a time interval, respectively. Constraint (11a) describes the flight direction of T-UAV. Constraint (11b)–(11d) restrict the trajectory of T-UAV according to the

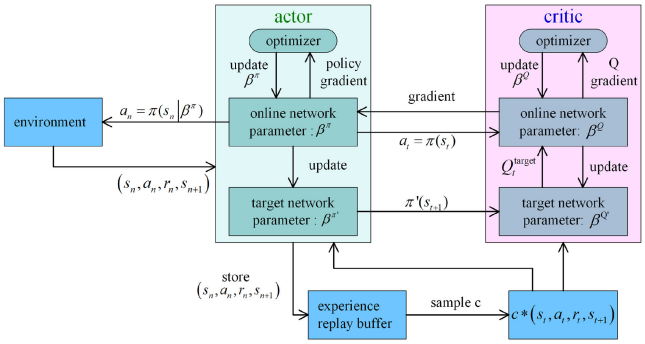


Fig. 3. DDPG framework.

T-UAV flight capacity. Constraint (11e)–(11f) describes the offloading decision of computing tasks whether B-UAVs offload tasks to T-UAV or not. Constraint (11g) restricts the number of VMs. It is obvious that this minimization is a non-convex and mixed integer nonlinear optimization problem, which is difficult to solve by traditional approaches. Therefore, in the following, we will propose a DRL approach to find our solution, since DRL is a powerful tool and suitable to deal with this complicated optimization problem with multiple non-convex constraints.

## IV. Proposed Algorithm

In this section, we present a DDPG algorithm for the optimal trajectory design and the configuration of VMs for T-UAV who works as an agent.

### A. Principle of DDPG

In traditional RL methods, an agent often selects an optimal action for each state of a system through continuous interaction with environment, so as to maximize a long-term cumulative reward rather than an instant one [12]. RL is a kind of learning from one's own experience, and is a suitable and powerful tool for automatic control and decision-making problems in random dynamic environment. The environment for RL is usually described as a theoretical framework based on discrete time Markov decision process (MDP). More specifically, according to Markov characteristic, an agent selects an action based on current state of environment, and at the same time he obtains an instant reward as a feedback to adjust next decision of the agent to maximize an accumulated reward. With the latest progress in machine learning, a beneficial combination of DNN and RL, namely DRL, is proposed [33]. Compared with traditional RL, DRL develops the ability of DNN to estimate correlation function in RL, thereby promoting more fast convergence and accurate approximation. Since DRL has greater potential to solve very complicated optimization problems in different application areas, we propose our DDPG algorithm based on DRL under an actor-critic framework as shown in Fig. 3.

DDPG uses a deterministic action strategy to output specific behavior rather than probability [39]. A deterministic strategy $\pi$ can be defined as a function

$$a_n = \pi(s_n|\beta^{\pi}) \tag{12}$$

where current state $s_n$ is its input, deterministic action $a_n$ is its output, and $\beta^{\pi}$ is the parameter of the actor network in this actor-critic framework.

The critic network donated as $Q(s_n, a_n|\beta^{Q})$ is used to approximate an action-value function under the actor policy $\pi(s_n|\beta^{\pi})$. There is also the parameter $\theta^{Q}$ in the critic network under the framework of the actor-critic algorithm [28]. Moreover, DDPG introduces an experience replay mechanism which randomly samples $c$ experiences from an experience pool $C$. The random samples eliminate the correlation and dependence among samples to make the algorithm easier to converge. Both actor and critic use a dual neural network architecture (target network and online network). The target networks of actor and critic are to update the approximated $\pi'(s_n|\beta^{\pi'})$ and $Q'(s_n, a_n|\beta^{Q'})$. The online networks of actor and critic update $\pi(s_n|\beta^{\pi})$ and $Q(s_n, a_n|\beta^{Q})$.

In DDPG, an objective function is defined as an expectation of a discounted cumulative reward with a normal form

$$J(\pi) = \mathbb{E}_{\beta^{\pi}}\left[r_0 + \gamma r_1 + \gamma^2 r_2 + \cdots + \gamma^n r_n\right] \tag{13}$$

where $\gamma \in [0, 1]$ is the discount rate. Specifically, we can minimize the following loss function to update a critic network

$$L(\beta^{Q}) = \frac{1}{c}\sum_{t=1}^{c}\left[Q_t^{\text{target}} - Q(s_t, a_t\left|\beta^{Q}\right.)\right]^2 \tag{14}$$

where $\mathrm{Q}_t^{\text{target}} = r_t + \gamma Q'(s_{t+1}, \pi'\left(s_t|\beta^{\pi'}\right)|\beta^{Q'})$ is the update target. In addition, the actor network is updated by the gradient strategy algorithm

$$\nabla_{\beta^{\pi}} J = \mathbb{E}_{s_t\sim\rho^{\theta}}\left[\nabla_a Q\left(s, a\left|\beta^{Q}\right.\right)\Big|_{\substack{s=s_t\\ a=\pi(s_t)}} \nabla_{\beta^{\pi}}\pi\left(s\left|\beta^{\pi}\right.\right)|_{s=s_t}\right] \tag{15}$$

where $\rho^{\theta}$ represents the distribution of $s_t$. We can update the parameters of the target network by

$$\begin{aligned}\beta^{Q'} &\leftarrow \tau\beta^{Q} + (1-\tau)\,\beta^{Q'}\\ \beta^{\pi'} &\leftarrow \tau\beta^{\pi} + (1-\tau)\,\beta^{\pi'}\end{aligned} \tag{16}$$

where $\tau \in (0, 1)$ represents the network update rate.

### B. Environment

In our model, the trajectory selection of T-UAV has Markov characteristics, hence DRL method can be used to optimize the trajectory in continuous state and action space. Normally, a MDP can be composed of four tuples $(S, A, R, P)$, where $S$ represents a set of possible states, $A$ represents a set of available actions, $P$ represents a set of required transition probability, and $R$ is a set of interested reward function. However, we do not use the state transition probability in this model. We redefine a MDP for our problem by three tuples as follows.

- State space: The state space of our scenario includes the offloaded data sizes of B-UAVs $\{L_{i,n}\}_{i\in M}$ and the locations of T-UAV $[X_n, Y_n]$ in time slot $n$, respectively. Thus, the state space $s_n$ can be defined as

$$s_n = \left\{\{L_{i,n}\}_{i\in\mathcal{M}}, [X_n, Y_n]\right\}_{n\in\mathcal{N}} \tag{17}$$

- Action space: A action $a_n$ for the UAV trajectory design and computing resource allocation consists of the flying direction and distance of T-UAV $\{\theta_n, l_n\}$, and the number of VMs $S_n$. Therefore, the action space can be formulated as

$$a_n = \{\{\theta_n, l_n\}, S_n\}_{n \in N} \tag{18}$$

- Reward Design: A reward is used to evaluate the quality of the chosen action. At each time slot, the number of B-UAVs to be offloaded to T-UAV is random. In order to achieve a better training effect, we take an average latency as our optimization goal, i.e.

$$T_n = \frac{1}{\sum_{i=1}^{M} b_{i,n}} \left(T_n^{tran} + \alpha T_n^{comp}\right) \tag{19}$$

where $0 < \alpha \leq 1$ is a weight factor to balance the transmission latency and computing latency. We introduce two penalty rewards $p_n^1$ and $p_n^2$ to restrain actions, where $p_n^1$ is the penalty constant incurred if T-UAV flies out of the target area and $p_n^2$ is the penalty constant incurred if the number of VMs is greater than the number of B-UAVs to be offloaded. Finally, we transform the objective of $\mathbf{OP}1$ into the accumulated reward maximization with the same constrants of $\mathbf{OP}1$.

$$\mathbf{OP2}: \max_{\mathbf{Q},\mathbf{K}} \sum_{n=1}^{N} r_n \tag{20}$$
$$s.t. \quad (11a) \sim (11g)$$

where $r_n = -T_n + p_n^1 + p_n^2$.

### C. Optimization Algorithms

In the following, two different DRL based algorithms, DQN and DDPG, are proposed in order to solve $\mathbf{OP}2$. DQN algorithm is mainly suitable for low dimension and discrete space optimization problem, while DDPG algorithm is better for high dimension and continuous space optimization problem. We provide these two algorithms in order to demonstrate that both can well deal with our specified problem with a little different performance.

*1) DQN Algorithm:* Our proposed DQN algorithm which combines a Q-learning algorithm and a neural network [40], is given as Algorithm 1. This algorithm predicts Q values by using a neural network whose inputs are the states and actions and outputs are the probability of each action respectively. Then, we minimize the following loss function of the neural network

$$L(\boldsymbol{\omega}) = \frac{1}{c} \sum_{t=1}^{c} [Q_t^{\text{target}} - Q(s_t, a_t \,|\boldsymbol{\omega})]^2 \tag{21}$$

where $\mathrm{Q}_t^{\text{target}} = r_t + \gamma \max_{a_{t+1}} Q'(s_{t+1}, a_{t+1}|\boldsymbol{\omega}')$ is the update target, $Q(s_t, a_t|\boldsymbol{\omega})$ is the Q estimate of the current state, $\boldsymbol{\omega}'$ and $\boldsymbol{\omega}$ are the weight vectors of the target neural network and the estimated neural network respectively, and $c$ is the number of samples from the experience pool $C$. In addition, we adopt the gradient descent method to update $\boldsymbol{\omega}$ of the neural network. In this way, we will find our solution for $\mathbf{OP}2$.

**Algorithm 1:** DQN-based Dynamic Trajectory Planning and Computation Resource Allocation Algorithm.

```
Initialize the estimated network with weights ω and the
  target network with weights ω'.
Initialize the experience replay buffer C.
for each episode do
  Initialize the locations of UAVs.
  for n = 1, 2, ..., N do
    Take action a_n = arg max_a Q(s_n, a|ω) or
      randomly choose an action with a certain
      probability.
    Update the state s_{n+1} and obtain the reward r_n.
    Store data (s_n, a_n, r_n, s_{n+1}) in the experience
      replay buffer C.
    if C is full, do
      Sample several random minibatches of c from C.
      Update the estimated network by (21).
      Update the target network with ω' ← ω.
  end for
end for
Return w.
Choose optimal action a_n^* = arg max_a Q(s_n, a, ω) at
  time n.
```

*2) DDPG Algorithm:* The given DDPG algorithm is named Algorithm 2, which is a DRL based algorithm under actor-critic architecture. In this algorithm, in each time slot, T-UAV selects an action $a_n$ in the actor network by (12). In order to achieve a better exploration to prevent the agent from falling into the local optimal strategy, we add an exploration noise $N_n$ to the action (12) as

$$a_n = \pi(s_n \,|\beta^\pi) + N_n \tag{22}$$

Then, the environment executes the newly generated action $a_n$ and returns the reward $r_n$ and the next state $s_{n+1}$. The agent stores the transition tuple $(s_n, a_n, s_{n+1}, r_n)$ generated from the process of state transition from $s_n$ to $s_{n+1}$ in experience replay buffer $C$. The agent randomly samples $c$ transition experience samples from the replay buffer. The critic network is updated by minimizing the loss function (14), and the actor network is updated by (15). Finally, the target networks of actor network and critic network are updated at a rate of $\tau$ by (16).

### D. Complexity Analysis

In this section, we will give a brief analysis the computation complexity of the proposed DDPG and DQN algorithms in the training process. Usually the complexity of a neural network is expressed as the number of operations of the network model, which is determined by the dimensions of the input state and action, the number of neurons in each layer of the neural network, and the number of neural network layers. The numbers of DQN neural network layers and neurons in the $g$th layer are denoted by $G$ and $u_g$, respectively. After the DQN algorithm experiences $N$ time slots and $F$ episodes, its neural

**Algorithm 2:** DDPG-based Dynamic Trajectory Planning and Computation Resource Allocation Algorithm.

Initialize the networks, including actor network $\pi(s_n|\beta^\pi)$, the target actor network with weights $\beta^{\pi'} = \beta^\pi$, the critic network $Q(s_n, a_n|\beta^Q)$ and the target critic network with weights $\beta^{Q'} = \beta^Q$.
Initialize the experience replay buffer $C$.
**for** each episode **do**
  Initialize the locations of UAVs.
  **for** $n = 1, 2, \ldots, N$ **do**
    Take action $a_n = \pi(s_n|\beta^\pi) + N_n$.
    Update the state $s_{n+1}$ and obtain the reward $r_n$.
    Store data $(s_n, a_n, r_n, s_{n+1})$ in the experience replay buffer $C$.
    **if** $C$ is full, **do**
      Sample several random minibatches of $c$ from $C$.
      Update the critic network $\beta^\pi$ by minimizing the critic loss (14).
      Update the actor network $\beta^Q$ by the gradient strategy algorithm (15).
      Update the target network by (16).
  **end for**
**end for**
Return $\beta^\pi$.
Choose optimal action $a_n^* = \pi(s_n|\beta^\pi)$ at time $n$.

network parameters converge and the complexity of the DQN neural network can be expressed as $\mathcal{O}(NF(\sum_{g=0}^{G-1} u_g u_{g+1}))$. Different from that of the DQN neural network, the computation complexity of the DDPG neural network needs to consider both actor and critic networks. The layer numbers of actor and critic neural networks are denoted by $G^a$ and $G^c$, respectively. In the $g$th layer, $u_g^{actor}$ and $u_g^{critic}$ represent the neuron numbers in the actor and critic networks, respectively. When the DDPG neural network parameters converge after $N$ time slots and $F$ episodes, its complexity is about $\mathcal{O}(NF(\sum_{g=0}^{G^a-1} u_g^{actor} u_{g+1}^{actor} + \sum_{g=0}^{G^c-1} u_g^{critic} u_{g+1}^{critic}))$. Compared with the DQN algorithm, this complexity is relatively larger.

Both DQN and DDPG algorithms are the successful combinations of certain neural networks and RL, and both use experience replay and target networks to solve the problems of algorithm instability and convergence difficulty. The important difference is that the action space of DQN algorithm is discrete. In the concern of high-dimensional problem with continuous action space, DQN algorithm quantizes continuous values into finite discrete values for training. However, this approximation may affect the training effect and the accumulated reward is relatively low. On the contrary, DDPG can handle continuous action space and large-dimensional state space, which is an unique advantage of the DDPG algorithm under the actor-critic framework.

DQN algorithm has only one neural network, namely value function network, which uses a random policy for training. DDPG algorithm adds a neural network on the basis of DQN algorithm, in which there are a value function network (critic network) and a policy network (actor network) in the training process. Therefore, the convergence speed of DQN algorithm is much faster than that of DDPG algorithm, and it is less complicated in realization.

TABLE I
SIMULATION PARAMETERS

| Notation | Physical meaning | Value |
|---|---|---|
| $B$ | Bandwidth | $1MHz$ |
| $P$ | Transmission power | $20dBm$ |
| $\delta$ | Length of each slot | $1s$ |
| $v_{\max}$ | Maximum flight speed | $150\mathrm{m/s}$ |
| $\rho_0$ | Channel power gain | $1.42 \times 10^{-4}$ |
| $\sigma_0^2$ | Power spectrum density | $-174\mathrm{dBm}$ |
| $L_{i,j,n}$ | Computation input data size | $[5, 7]\mathrm{Kb}$ |
| $L_{\max}$ | B-UAV queue reach the limited size | $1Mb$ |
| $H$ | Height of T-UAV | $1500m$ |
| $D$ | Degradation factor | 0.2 |
| $r$ | Computation rate of a VM | $5 \times 10^5 \mathrm{bits/s}$ |
| $\alpha$ | Weight factor | 0.5 |
| $C$ | Experience replay buffer | 200000 |
| $c$ | Batch size | 32 |

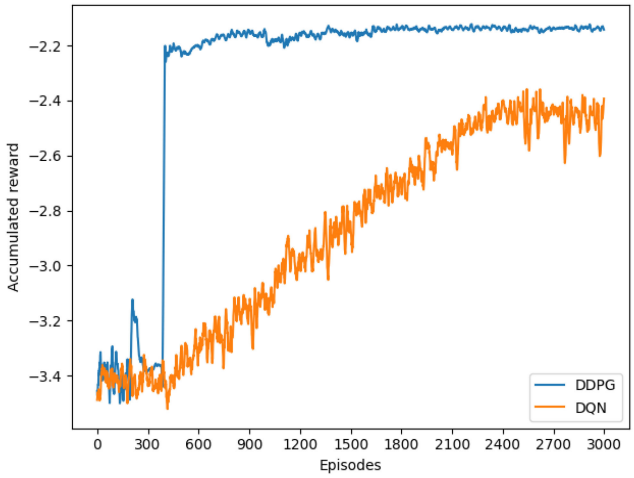


Fig. 4. Accumulated reward of DDPG and DQN algorithms.

## V. PERFORMANCE EVALUATION

In this section, we evaluate the performance of the proposed DDPG and DQN algorithms. The simulations are conducted by using Python 3.6 and TensorFlow 1.14.0. In the simulations of DDPG algorithm, two hidden layers with [400, 450] neurons in both actor and critic networks are used for training. The learning rate of actor network is $5 \times 10^{-4}$, and the learning rate of critic network is $3 \times 10^{-4}$. In the simulations of DQN algorithm, two hidden layers with [400, 450] neurons in neural network is used for training. The learning rate of neural network is $1 \times 10^{-4}$. In our simulations, we set 6000 m $\times$ 6000 m target area. We set the coordinates of B-UAVs as [1000, 1000, 500], [3000, 1000, 500], [5000, 1000, 500], [1000, 3000, 500], [3000, 3000, 500], [5000, 3000, 500], [1000, 5000, 500], [3000, 5000, 500], [5000, 5000, 500] m, respectively. The initial location of T-UAV is [3000, 3000, 1500] m. The rest of parameters are shown in Table I.

First, we present the accumulated rewards of DDPG and DQN algorithms shown in Fig. 4. It shows that the accumulated rewards of two algorithms fluctuate at a very low values at

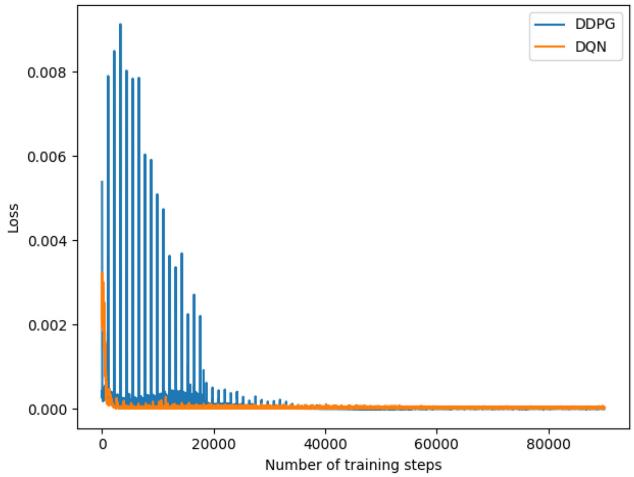


Fig. 5. Convergence of DDPG and DQN algorithms.

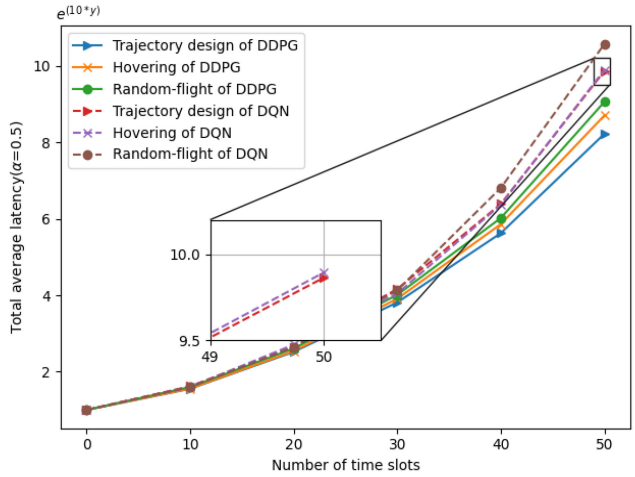


Fig. 6. Total average latency comparison among different trajectory designed by DDPG and DQN algorithms.

the beginning, because in the initial experience stage, T-UAV does not have any environment knowledge, and the action is almost randomly selected. T-UAV begins to train the network using the accumulated samples when it accumulates enough samples within certain time. At this stage, we can see that the accumulated reward of DDPG algorithm is significantly higher than that of DQN algorithm. The accumulated reward of DDPG algorithm rises rapidly during the training process, and finally converges to a high value, which implies the best flight strategy is obtained. However, the accumulated reward of DQN algorithm rises slowly during the training process, and it converge to a lower value until about 2400 episodes of training. As mentioned earlier, because DDPG algorithm is based on actor-critic framework, it can process continuous inputs and output continuous action, while DQN algorithm divides continuous inputs into discrete inputs and outputs the probability of each action. Therefore, when facing high-dimensional inputs, the training effect of DQN algorithm is less better than DDPG algorithm. However, the convergence speed of DQN algorithm is significantly faster than that of DDPG algorithm shown in Fig. 5.

In Fig. 5 we can see that DQN algorithm converges after about 20000 steps, while DDPG algorithm converges after about 40000 steps. This is because DQN algorithm has only one neural network set, and DDPG algorithm has two neural network sets to be learned, namely the actor neural network and the critic neural network. Although the loss function of DDPG algorithm converges to a small and stable value slowly, it has been explored more extensively in the initial stage, thus this algorithm shows better performance after enough training. However, the loss function of DQN algorithm converges to a small and stable value faster. If we consider an optimization problem with discrete and small-dimensional inputs, DQN algorithm is still a suitable solution.

In Fig. 6, we compare the total average latencies when using the trajectory designs of DDPG and DQN algorithms with those of taking other trajectory selections by T-UAV, such as hovering and random-flight defined as follows.

Hovering: T-UAV hovers in the center of the target area to help B-UAVs offload and calculate tasks.

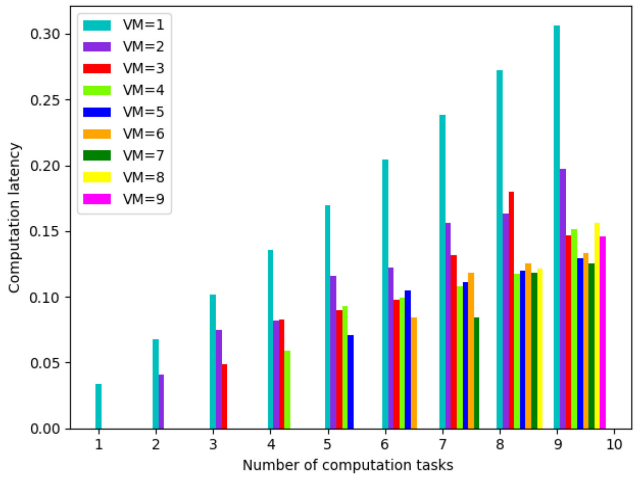


Fig. 7. Computation latency in different number of VMs with same data size for all tasks.

Random-flight: T-UAV flies randomly in the target area.

In order to observe the difference after the above trajectory selections, we perform a set of experiment on total average latency. In Fig. 6, we find that as the time slot increases, the total average latency based on the trajectory design is lower than those of hovering and random-flight strategies by using both DDPG and DQN algorithms. This is because the trajectory design can intelligently adjust the position of T-UAV according to the requirements and the positions of different B-UAVs in each time slot. However, the latency performance of DQN algorithm is not as good as those of DDPG algorithm due to the discrete space limitation. The all total average latencies are greater than the corresponding latencies produced by DDPG algorithm for three trajectory selections. Therefore, DDPG algorithm has better result to reduce the latency through the trajectory design. This result proves that the trajectory designed by DDPG algorithm is superior and effective.

Fig. 7 shows that the effect of the number of VMs on the total computation latency when the data size of all tasks is the same. We find that as the number of tasks increases, the increase of the

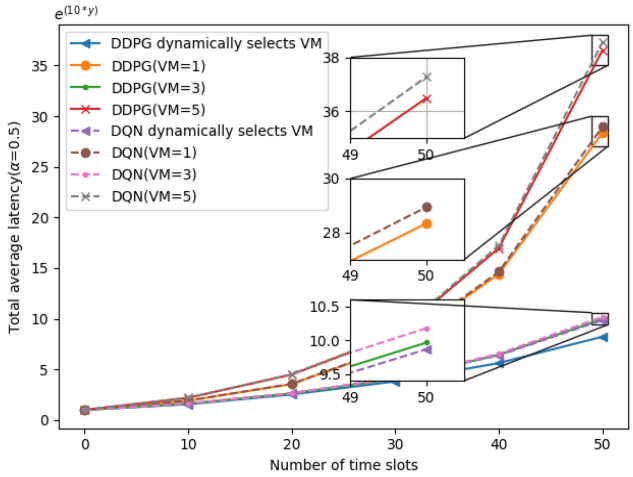


Fig. 8. Total average latency comparison among different number of VMs by DDPG and DQN algorithms.

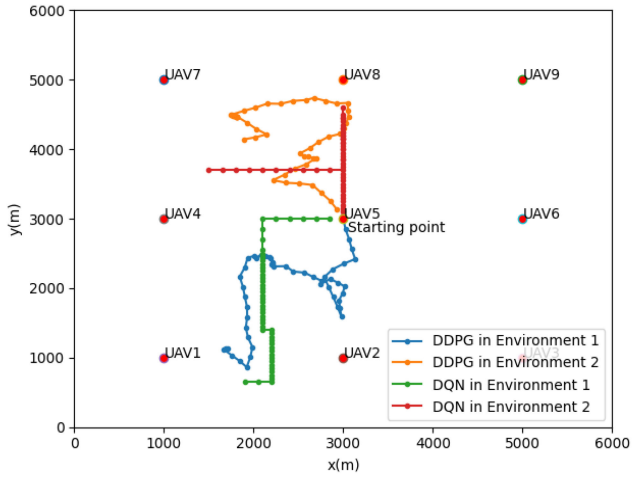


Fig. 9. T-UAV trajectories under different environment by DDPG and DQN algorithms.

number of VMs can significantly reduce computation time, but it is not the more the better. We observe that when the number of VMs is large enough, the latency performance will drop quickly due to the I/O interference between VMs. Since the task data size is random in our system, traditional methods are difficult to predict the optimal number of VMs. Therefore, we use a DRL based method to dynamically select the number of VMs in each time slot to achieve better results.

In Fig. 8, we compare the total average latencies in consideration of the dynamic selection of the number of VMs by using different algorithms with those of three cases that the number of fixed VMs is 1, 3 and 5 respectively. The results show that both DDPG and DQN algorithms can clearly reduce the total average latencies by the dynamic selection of the number of VMs. When the number of VMs is fixed, the total average latencies of DDPG algorithm is slightly lower than the corresponding latencies of DQN algorithm. This is because when the number of VMs is fixed, the computation latency is relatively large, and the communication latency accounts for a relatively small proportion of the total average latency. However, it is not obvious to reduce the latency performance through trajectory optimization by using DDPG and DQN algorithms. We can also see that the total average latencies when the fixed number of VMs is 1 is greater than those when the fixed number of VMs is 3. And the total average latencies when the fixed number of VMs is 5 is greater than those when the fixed number of VMs is 3. It indicates that a reasonable choice of the number of VMs is important to reduce the total average latency. In summery, both trajectory optimization and configuration of VM can reduce the total average latency, but the reasonable multiplexed VMs is more effective in reducing latency compared to trajectory design.

In order to further prove the effectiveness of our DDPG and DQN algorithms, Fig. 9 shows the trajectories of T-UAV under these two algorithms in different random communication environment under large coverage. Environment 1 is the case where only B-UAV1 to B-UAV 6 may be needed to offload tasks to T-UAV in mission-intensive area. Environment 2 is the case where only B-UAV4 to B-UAV 9 may be needed to offload tasks to T-UAV in mission-intensive area. The simulation results using two algorithms in two cases show that T-UAV can approach the target area when only part of B-UAVs offload the tasks, with the trajectory of T-UAV to minimize the total average latency. In the same case, the trajectory of DDPG algorithm after training has a lower latency than that of DQN algorithm. This is because that the action space of DDPG algorithm is continuous, and the choice of flight direction and flight distance can be any value that satisfies the constraints. Therefore, this algorithm can dynamically change the current flight direction and flight distance of motion according to the requirements and the positions of different B-UAVs in each time slot. While DQN algorithm divides the continuous action space into a finite number of elements. In our simulations the flight direction is discretized into four directions, up, down, left, and right. Therefore, the trajectory of DDPG algorithm is more flexible and efficient than that of DQN algorithm, even though we can make more elements for the flight direction, which may result in better latency performance.

The weight factor determines the relationship between the communication latency and computation latency by (19), thereby it affects the system optimization decision. We make several possible choices for $\alpha$ to obtain the total average latencies under different algorithms shown in Fig. 10. We find that when $\alpha = 0.5$, the latencies of both two algorithms are about the smallest respectively. Different choice of $\alpha$ means that the system pays different attention on communication latency and computation latency, which is a trade-off between the costs of communication and computation. According to (19), we know that the varied reward function of the system will lead to different optimization strategies. In general, the latency performance of DDPG algorithm is better than that of DQN algorithm in all aspects but it is more complicated in realization.

Finally, we compare DDPG and DQN algorithms with other baseline algorithms to show the advantages of our proposed two algorithms. Fig. 11 shows the total average latency of different algorithms, including three benchmark policies: greedy, conservative and random. We can see that the total average latency

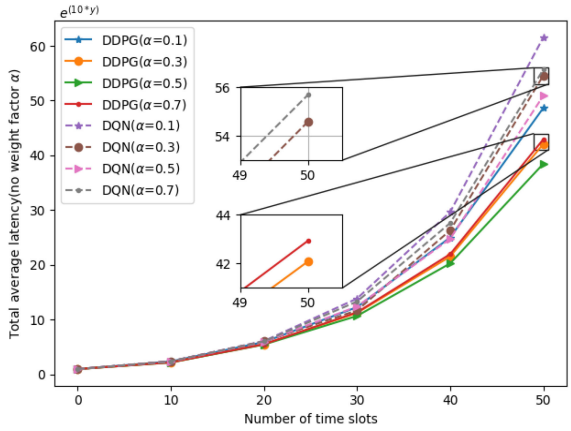


Fig. 10. Total average latency vs $\alpha$ by DDPG and DQN algorithms.

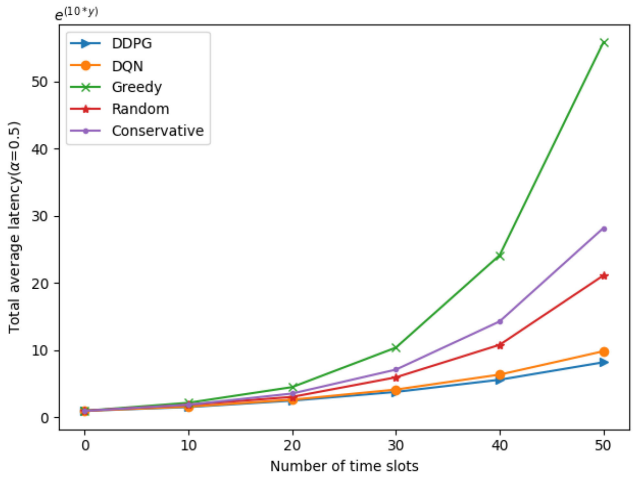


Fig. 11. Total average latency comparison among different algorithms.

of DDPG and DQN algorithms are always better than these traditional algorithms. As mentioned before, since the DDPG algorithm does not have quantization error, it shows better performance than DQN algorithm. For the greedy algorithm, the hovering T-UAV configures as much VMs as possible in each time slot to reduce its computing latency, however it does not consider overall performance degradation due to the I/O interference between VMs. For the conservative algorithm, in order to prevent the situation of too large overall performance degradation, the hovering T-UAV only configures a fixed number of VMs in each time slot. While for the random algorithm, the hovering T-UAV randomly configures the number of VMs. Since the RL-based algorithms can intelligently adjust the number of VMs according to the number of offloading tasks, the total average latency of DDPG and DQN algorithms are much smaller than those of the greedy, conservative and random algorithms.

## VI. Conclusion

Facing grand challenges of rapid growth and computational intensiveness in marine wireless communication networks, this article provides two optimal solutions to provide powerful computing capabilities for resource shortage or delay-sensitive marine communication terminals. The formulated joint optimization of the trajectory design and configuration selection of VMs of T-UAV is a non-convex problem, which is difficult to solve directly. A DQN algorithm and a DDPG algorithm based on DRL are proposed to find optimal flight trajectories and the number of VMs participating in parallel computing for T-UAV. The simulation results show that compared to a case of hovering in the center and no parallel computing, our DDPG algorithm reduces the total average latency by more than 37% through jointly optimizing the trajectory of T-UAV and the configuration of the number of VMs, while DQN algorithm reduces it by 31%. Both DQN algorithm and DDPG algorithm show good training effects in our system, which greatly improve the latency performance. However, it is obvious that DDPG algorithm has better latency performance improvement than DQN algorithm for joint trajectory optimization and configuration selection of VMs due to the discrete space limitation of DQN algorithm. Therefore, for the optimization problem with higher dimension and continuous action spaces, DDPG algorithm can choose better actions without quantization errors to enhance the latency performance at the cost of heavy computation.

Moreover, the DRL based optimization methods are more robust to the problem modelling, since they are not sensitive to the convexity of the mathematical models and the constraints of the problems, which gives us more possibility to solve complicated optimization problems in wireless communication scenarios. However, these methods often need more computation capability, which means strong hardware support and development.

This article only uses random user requests and simplified channel conditions. In the future work, we will consider possible intelligent reception and transmission prediction based on DRL to actively predict user requests and time-varying channel conditions, thereby developing more effective trajectory optimization and resource allocation scheme.

## References

[1] Y. Huo, X. Dong, and S. Beatty, "Cellular communications in ocean waves for maritime Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 10, pp. 9965–9979, Oct. 2020.

[2] V. Fontana, J. M. D. Blasco, A. Cavallini, N. Lorusso, A. Scremin, and A. Romeo, "Artificial intelligence technologies for Maritime Surveillance applications," in *Proc. 21st IEEE Int. Conf. Mobile Data Manage.*, 2020, pp. 299–303.

[3] L. Liu, Q. Gu, L. Li, and X. Lai, "Research on maritime search and rescue recognition based on agent technology," in *Proc. Int. Conf. Artif. Intell. Electromechanical Automat.*, 2020, pp. 201–205.

[4] X. Li, W. Feng, Y. Chen, C. Wang, and N. Ge, "UAV-Enabled accompanying coverage for hybrid satellite-UAV-terrestrial maritime communications," in *Proc. 28th Wireless Opt. Commun. Conf.*, 2019, pp. 1–5.

[5] X. Li, W. Feng, Y. Chen, C. Wang, and N. Ge, "Maritime coverage enhancement using UAVs coordinated with hybrid satellite-terrestrial networks," *IEEE Trans. Commun.*, vol. 68, no. 4, pp. 2355–2369, Apr. 2020.

[6] T. Wei, W. Feng, J. Wang, N. Ge, and J. Lu, "Exploiting the shipping lane information for energy-efficient maritime communications," *IEEE Trans. Veh. Technol.*, vol. 68, no. 7, pp. 7204–7208, Jul. 2019.

[7] F. Xu, F. Yang, C. Zhao, and S. Wu, "Deep reinforcement learning based joint edge resource management in maritime network," *China Commun.*, vol. 17, no. 5, pp. 211–222, May 2020.

[8] S. Jiang, "A possible development of marine Internet: A large scale cooperative heterogeneous wireless network," in *Internet of Things, Smart Spaces, and Next Generation Networks and Systems*. Cham: Springer, 2015, pp. 481–495.

[9] Y. Xu, "Quality of service provisions for maritime communications based on cellular networks," *IEEE Access*, vol. 5, pp. 23881–23890, 2017.

[10] R. Campos, T. Oliveira, N. Cruz, A. Matos, and J. M. Almeida, "BLUECOM+: Cost-effective broadband communications at remote ocean areas," in *Proc. OCEANS 2016 - Shanghai*, 2016, pp. 1–6.
[11] T. Yang, H. Feng, C. Yang, Y. Wang, J. Dong, and M. Xia, "Multivessel computation offloading in maritime mobile edge computing network," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4063–4073, Jun. 2019.
[12] B. Cao, L. Zhang, Y. Li, D. Feng, and W. Cao, "Intelligent offloading in multi-access edge computing: A. state-of-the-art review and framework," *IEEE Commun. Mag.*, vol. 57, no. 3, pp. 56–62, Mar. 2019.
[13] G. Gakpo, X. Su, and C. Choi, "Moving intelligence of mobile edge computing to maritime network," in *Proc. Conf. Res. Adaptive Convergent Syst.*, 2019, pp. 189–193.
[14] X. Pu, L. Liu, Y. Mei, S. Sivathanu, Y. Koh, and C. Pu, "Understanding performance interference of I/O workload in virtualized cloud environments," in *Proc. IEEE 3rd Int. Conf. Cloud Comput.*, 2010, pp. 51–58.
[15] D. Bruneo, "A stochastic model to investigate data center performance and QoS in IaaS cloud computing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 3, pp. 560–569, Mar. 2014.
[16] Z. Liang, Y. Liu, T. Lok, and K. Huang, "Multiuser computation offloading and downloading for edge computing with virtualization," *IEEE Trans. Wireless Commun.*, vol. 18, no. 9, pp. 4298–4311, Sep. 2019.
[17] H. Wang, Y. Wang, Y. Ma, and B. Lin, "Resource allocation for OFDM-based maritime edge computing networks," in *Proc. 13th Int. Congr. Image Signal Process., BioMed. Eng. Inform.*, 2020, pp. 983–988.
[18] Q. Hu, Y. Cai, G. Yu, Z. Qin, M. Zhao, and G. Y. Li, "Joint offloading and trajectory design for UAV-Enabled mobile edge computing systems," *IEEE Internet Things J.*, vol. 6, no. 2, pp. 1879–1892, Apr. 2019.
[19] M. Li, N. Cheng, J. Gao, Y. Wang, L. Zhao, and X. Shen, "Energy-efficient UAV-Assisted mobile edge computing: Resource allocation and trajectory optimization," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3424–3438, Mar. 2020.
[20] Z. Yang, C. Pan, K. Wang, and M. Shikh-Bahaei, "Energy efficient resource allocation in UAV-Enabled mobile edge computing networks," *IEEE Trans. Wireless Commun.*, vol. 18, no. 9, pp. 4576–4589, Sep. 2019.
[21] J. Zhang *et al.*, "Computation-efficient offloading and trajectory scheduling for Multi-UAV assisted mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 2, pp. 2114–2125, Feb. 2020.
[22] T. Wei, W. Feng, Y. Chen, C. X. Wang, N. Ge, and J. Lu, "Hybrid satellite-terrestrial communication networks for the maritime Internet of Things: Key technologies, opportunities, and challenges," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 8910–8934, Jun. 2021.
[23] Y. Pang, D. Wang, D. Wang, L. Guan, C. Zhang, and M. Zhang, "A space-air-ground integrated network assisted maritime communication network based on mobile edge computing," in *Proc. IEEE World Congr. Serv.*, 2020, pp. 269–274.
[24] C. Xu *et al.*, "Sixty years of coherent versus non-coherent tradeoffs and the road from 5G to wireless futures," *IEEE Access*, vol. 7, pp. 178246–178299, 2019.
[25] S. Yin, S. Zhao, Y. Zhao, and F. R. Yu, "Intelligent trajectory design in UAV-Aided communications with reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 68, no. 8, pp. 8227–8231, Aug. 2019.
[26] A. M. Koushik, F. Hu, and S. Kumar, "Deep $Q$ -learning-based node positioning for throughput-optimal communications in dynamic UAV swarm network," *IEEE Trans. Cogn. Commun. Netw.*, vol. 5, no. 3, pp. 554–566, Sep. 2019.
[27] H. Van Hasselt, A. Guez, and D. Silver, "Deep reinforcement learning with double Q-learning," in *Proc. AAAI Conf. Artif. Intell.*, 2016, vol. 30, no. 1.
[28] R. Ding, F. Gao, and X. S. Shen, "3D UAV trajectory design and frequency band allocation for energy-efficient and fair communication: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 12, pp. 7796–7809, Dec. 2020.
[29] A. M. Seid, G. O. Boateng, S. Anokye, T. Kwantwi, G. Sun, and G. Liu, "Collaborative computation offloading and resource allocation in Multi-UAV assisted IoT networks: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 8, no. 15, pp. 12 203–12 218, Aug. 2021.
[30] J. Sun, G. Gui, H. Sari, H. Gacanin, and F. Adachi, "Aviation data lake: Using side information to enhance future air-ground vehicle networks," *IEEE Veh. Technol. Mag.*, vol. 16, no. 1, pp. 40–48, Mar. 2021.
[31] G. Gui, M. Liu, F. Tang, N. Kato, and F. Adachi, "6G: Opening new horizons for integration of comfort, security, and intelligence," *IEEE Wireless Commun.*, vol. 27, no. 5, pp. 126–132, Oct. 2020.
[32] M. Alzenad, A. El-Keyi, and H. Yanikomeroglu, "3-D placement of an unmanned aerial vehicle base station for maximum coverage of users with different QoS requirements," *IEEE Wireless Commun. Lett.*, vol. 7, no. 1, pp. 38–41, Feb. 2018.
[33] L. Wang, K. Wang, C. Pan, W. Xu, N. Aslam, and L. Hanzo, "Multi-agent deep reinforcement learning based trajectory planning for Multi-UAV assisted mobile edge computing," *IEEE Trans. Cogn. Commun. Netw.*, vol. 7, no. 1, pp. 73–84, Mar. 2021.
[34] M. D. Nguyen, T. M. Ho, L. B. Le, and A. Girard, "UAV trajectory and sub-channel assignment for UAV based wireless networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2020, pp. 1–6.
[35] Z. Yu, Y. Gong, S. Gong, and Y. Guo, "Joint task offloading and resource allocation in UAV-Enabled mobile edge computing," *IEEE Internet Things J.*, vol. 7, no. 4, pp. 3147–3159, Apr. 2020.
[36] J. Zong *et al.*, "Flight time minimization via UAV's trajectory design for ground sensor data collection," in *Proc. 16th Int. Symp. Wireless Commun. Syst.*, 2019, pp. 255–259.
[37] Q. Zhang, J. Chen, L. Ji, Z. Feng, Z. Han, and Z. Chen, "Response delay optimization in mobile edge computing enabled UAV swarm," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3280–3295, Mar. 2020.
[38] Z. Ning, J. Huang, and X. Wang, "Vehicular fog computing: Enabling real-time traffic management for smart cities," *IEEE Wireless Commun.*, vol. 26, no. 1, pp. 87–93, Feb. 2019.
[39] B. Li and Y. Wu, "Path planning for UAV ground target tracking via deep reinforcement learning," *IEEE Access*, vol. 8, pp. 29 064–29 074, 2020.
[40] J. Zhang *et al.*, "Trajectory planning of UAV in wireless powered IoT system based on deep reinforcement learning," in *Proc. IEEE/CIC Int. Conf. Commun. China*, 2020, pp. 645–650.

**Ying Liu** received the bachelor's degree in communication engineering from the Changchun University of Science and Technology, Changchun, China, in 2017, and the master's degree from Communication engineering China, in 2022. Her research focuses on wireless communication in UAV swarm mobile edge computing network.

**Junjie Yan** (Student Member, IEEE) received the bachelor's degree in communication engineering in 2017 from the College of Communication Engineering, Jilin University, Changchun, China, where she is currently working toward the Ph.D. degree. Her research interests include intelligent edge computing, UAV communications, and deep reinforcement learning.

**Xiaohui Zhao** (Member, IEEE) received the Ph.D. degree in applied mathematics and control theory from the Université de Technologie de Compiègne, Compiègne, France, in 1993. In 2006, he was a Senior Visiting Scholar for half a year with the Laboratoire d'Informatique, Université de Pierre et Marie Curie, Paris, France. He is currently a Professor of communication engineering with Jilin University, Changchun, China. His research interests include wireless communication, cognitive radio, and adaptive signal processing.